

How to Train Your Mamba for Time Series Forecasting

Jiaxi Hu

The Hong Kong University of Science
and Technology (Guangzhou)
guangzhou, guangdong, China
jhu110@connect.hkust-gz.edu.cn

Disen Lan

Fudan University
Shanghai, China
disenlan1002@gmail.com

Ziyu Zhou

DSA Thrust, The Hong Kong
University of Science and Technology
(Guangzhou)
guangzhou, guangdong, China
zzhou651@connect.hkust-gz.edu.cn

Gefeng Luo

Information Hub, The Hong Kong
University of Science and Technology
(Guangzhou)
guangzhou, guangdong, China
gluo812@connect.hkust-gz.edu.cn

Qingsong Wen

AI Research Institute, Squirrel AI
Learning
Seattle, WA, USA
qingsongedu@gmail.com

Yuxuan Liang*

INTR & DSA, The Hong Kong
University of Science and Technology
(Guangzhou)
guangzhou, guangdong, China
yuxliang@outlook.com

Abstract

State Space Models (SSMs) have emerged as a powerful framework for sequence modeling in recent years. By approximating continuous dynamical systems and applying discretization techniques, SSMs are particularly well-suited for modeling time-series data. However, despite their growing popularity, most existing applications of SSMs in time-series forecasting treat the models as black boxes. Besides, the underlying mechanisms that contribute to their effectiveness remain unclear, and common claims regarding their advantages in efficiency and expressiveness are not fully substantiated. To address these gaps, this paper establishes a theoretical connection between SSMs and classical spectral transformations from signal processing, thereby providing a more interpretable foundation. Furthermore, we conduct comprehensive ablation studies to examine the properties of different SSM configurations. Our goal is to offer both theoretical insight and empirical guidance for future research on SSM-based approaches in time-series forecasting.

CCS Concepts

• Information systems → Spatial-temporal systems.

Keywords

State Space Model; Time Series Forecasting

ACM Reference Format:

Jiaxi Hu, Disen Lan, Ziyu Zhou, Gefeng Luo, Qingsong Wen, and Yuxuan Liang*. 2026. How to Train Your Mamba for Time Series Forecasting. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1 (KDD 2026)*, August 9–13, 2026, Jeju Island, Republic of Korea. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3770854.3780279>



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

KDD 2026, Jeju Island, Republic of Korea.

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2258-5/2026/08

<https://doi.org/10.1145/3770854.3780279>

1 Introduction

State Space Models (SSMs) [8, 10, 12] are recent deep learning models that have demonstrated significant potential in various sequence modeling tasks. Unlike Transformers, which perform weighted aggregation over all tokens simultaneously, these models process information recursively and generalize it for unseen data. Recently, a series of models [2, 3, 23, 31, 36] encapsulate the state-of-the-art variants (i.e., Mamba [10]) as black-box modules, attempting to enhance their temporal modeling ability.

Although these models show improved performance, the underlying mechanisms by which SSMs contribute to time series forecasting (TSF) [18] remain unclear. Moreover, the motivation behind these models for adopting Mamba is questionable in the realm of TSF, particularly with respect to training efficiency and expressive ability. Detailed analysis is provided below:

Is Mamba really efficient in TSF? Mamba is originally designed for language modeling tasks, with the goal of employing linear recurrence to address the unbounded growing KV cache memory and inference time in autoregressive Transformers [28]. However, due to error accumulation in TSF, predictions are typically made through linear projection instead of autoregressive decoding. As a result, Mamba’s advantages are difficult to realize.

Furthermore, although Mamba enjoys theoretically linear training complexity, modern hardware such as GPUs are inherently optimized for matrix multiplication (GEMM) by tensor cores [6, 21]. As a result, Mamba requires carefully engineered custom kernels and long sequences to achieve computational efficiency comparable to that of Transformers. In typical time series forecasting scenarios, the input sequence length rarely exceeds 720 time steps and often reduces to only tens of tokens when patching is applied (Even variable-wise, the count stays in the low hundreds). At such short sequence lengths¹, even a naïve PyTorch implementation of a Transformer surpasses Mamba in computational efficiency, not to mention highly optimized operators such as FlashAttention [6].

Is Mamba really expressive in TSF? The evolution of SSMs reveals a clear trajectory in the design of the state transition (that

¹Typically, FlashAttention is faster than Mamba within 8k and Mamba2 within 3k sequence length [16, 34]. This advantage becomes even more pronounced when training with low precision on Hopper-series GPUs [26].

is, the matrix A). Early works grounded in HiPPO theory [11] employ structured dense matrices; S4D [14] later reduces this to diagonal matrices; and Mamba2 [8] pushes the idea to the head-wise scalar transition. The motivation is computational: Computing a linear recurrence involves raising the transition matrix to high powers; when the matrix is dense, this is computationally expensive, whereas diagonal matrices only require powering the diagonal entries, and scalar allows for efficient chunk-wise parallelism [8, 34].

However, these speed-ups come at a price: a diagonal or scalar transition cannot express the rich dynamics that a dense matrix can. In large-scale language pretraining, raw efficiency outweighs the loss, and the gap can be offset by auxiliary mechanisms. For instance, Mamba inserts short convolutions to recover the low-rank components missing from the diagonal matrix, and Mamba2 enlarges the state dimension to compensate [16]. While in the lightweight TSF task, it remains unclear whether this reduction is justified.

This work aims to offer theoretical and empirical guidance on applying state space models for time series forecasting tasks. As immediate consequences of this paper:

- (Section 3) We argue that, rather than focusing on computational efficiency (which may be unrealistic) or marginal gains in empirical performance, the primary advantage of applying SSMs to time series lies in their strong theoretical grounding in signal processing. This foundation enables the development of interpretable forecasting models. To support this view, we establish a theoretical link between SSMs and spectral transformations.
- (Section 4) We argue that Mamba’s primary performance gains stem from its data-dependent selection mechanism. Building on it, we analyze the effectiveness of each module in Mamba and investigate the performance of various SSM variants with structured transition matrices in TSF.
- Our key insights include the following:

- i) For TSF, Mamba’s full architecture is not optimal; keeping only the time-varying SSM kernel yields better performance.
- ii) For TSF, a structured dense state-transition matrix outperforms the diagonal & scalar transition used in Mamba.
- iii) For TSF, sometimes a specialized initialization for SSM matrices ABC is essential, similar to [1].
- iv) For TSF, the patching operation is a local linear approximation of the underlying dynamical system, effectively increasing the expressive capacity of the SSM kernel.

2 Preliminary: SSM Kernel

Defined by Eq. 1, continuous-time SSMs are parameterized maps that expand the input $u(t)$ into an N -dimensional latent space and project it onto the output $y(t)$, which can be easily extended to multi-input/output [27] SSM with input $u \in D$ dimension.

$$x'(t) = Ax(t) + Bu(t), \quad y(t) = Cx(t) \quad (1)$$

LEMMA 2.1. *For a differential equation of $x'(t) = Ax(t) + Bu(t)$, its general solution can be computed as:*

$$x(t) = e^{A(t-t_0)}x(t_0) + \int_{t_0}^t e^{A(t-s)}Bu(s)ds.$$

The two terms in Lemma 2.1 correspond to the *zero-input response* and the *zero-state response* components, as defined in linear systems theory [32]. When the initial state is set to zero, that is, $x(t_0) = 0$, the state-space model (SSM) simplifies to a convolutional form, which is utilized for efficient computation by earlier models such as S4 [12]. However, recent studies [25] have demonstrated that retaining a nonzero initial state can improve the model’s ability to generalize over long sequences, which is typically achieved during training by initializing each batch with the final state of the previous one, sampled randomly. In this work, we adopt the standard setting with a zero initial state.

Hippo [11] theory provides a mathematical framework for deriving the ABC matrices: Given an input $u(s)$, a set of closed-recursive orthogonal basis $p_n(t, s)$ that $\int_{-\infty}^t p_m(t, s)p_n(t, s)ds = \delta_{m,n}$, and an inner product probability measure $\mu(t, s)$. As shown in Eq. 2, this enables us to project the input $u(s)$ onto the basis along the time direction in Hilbert space $\mathcal{H}_\mu$. By collecting all projection coefficients, the Equation 1 can be obtained, where the forward projection process corresponds to the AB matrices, and the backward reconstruction process corresponds to the C matrix.

$$x_n(t) = \langle u, p_n \rangle_\mu = \int_{-\infty}^t u(s)p_n(t, s)\omega(t, s)ds, \quad (2)$$

It is evident that the projection coefficients depend on both the terms $\omega(t, s)$ and $p_n(t, s)$. We can combine them into a single term $K_n(t, s) = p_n(t, s)\omega(t, s)$, called the SSM kernel. s and t denote the variables of the input and state function.

3 Theory: SSM as Spectral Operator

Based on the concept of the SSM kernel, a state space model can be viewed as a specialized neural operator, which projects the input signal into a compact spectral space.

$$\text{SSM} := \mathcal{K}u(s) = \int_0^1 K(t, s)u(s)ds \quad (3)$$

We show that, with an appropriately chosen set of orthogonal basis functions $p_n(t, s)$ and a corresponding inner-product measure $\omega(t, s)$, the state-space model (SSM) formulation can be reduced to several classical spectral transformation techniques, as illustrated in Fig. 1. These techniques have long been foundational in signal processing and time-series analysis.

i) **Fourier Transform.** can be realized with:

$$\omega(t, s) = \mathbf{1}_{[0,1]}, \quad p_n(t, s) = e^{-its}.$$

Where $\mathbf{1}$ denotes the domain indicator function. This particular system leverages the expressive capacity of trigonometric functions to encode the periodic components of a time series in the spectral domain, enabling efficient representation of global frequency patterns. Nonetheless, both $\omega(t)$ and $p_n(t, s)$ are defined globally over the entire temporal domain, which limits the model’s ability to capture localized dynamics. As a result, fine-grained temporal variations, such as localized events, or short-term dependencies, may not be adequately modeled within this framework.

ii) **Gabor Transform.** can be realized with:

$$\omega(t, s) = \mathbf{1}_{[0,1]}, \quad p_n(t, s) = g(t - \tau)\phi_n(t, s).$$

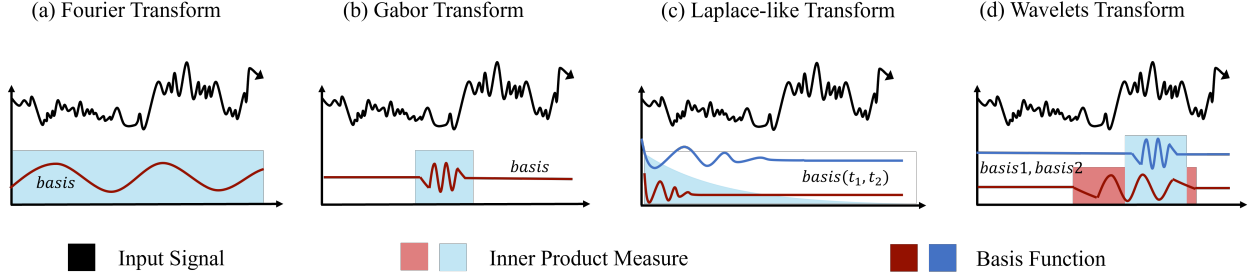


Figure 1: Classical spectral transform with basis function and inner product measure. (a) Fourier transform uses trigonometric basis functions with a global measure, making it well-suited for modeling periodicity. (b) The Gabor transform focuses on projecting the input within localized measurement windows. (c) The Laplace transform applies exponential decay, making it effective for capturing long-term dependencies. (d) Wavelet transform enables projections at multiple scales, allowing both local and global structure to be modeled.

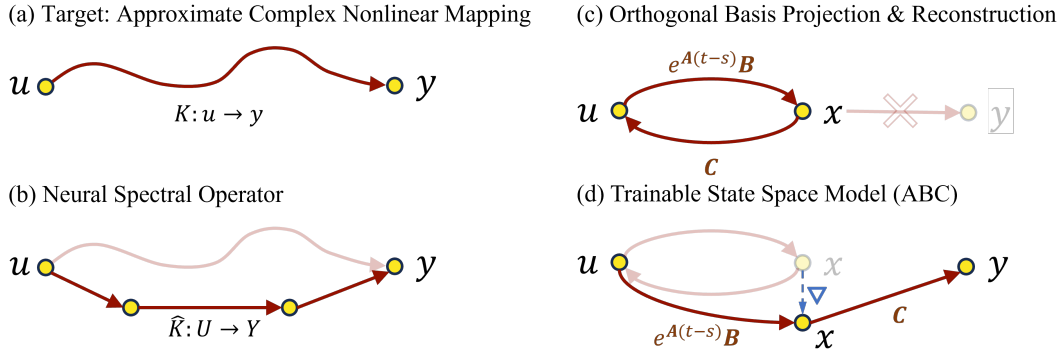


Figure 2: (a) In the TSF task, we aim to find a complex nonlinear mapping $\mathcal{K}$ from past observed data u to future predicted data y . (b) Neural operators simplify the parameterization of mapping $\mathcal{K}$ by leveraging an easily learnable spectral space. (c) SSM(AB) represents the projection and reconstruction to the spectral space x . (d) Mamba is a dynamic operator that gradually moves toward an optimal spectral space and finds a linear observation function.

The Gabor function $g(t - \tau)$ defines a localized window that truncates the orthogonal basis functions $\phi_n(t, s)$, thereby indirectly influencing the domain indicator $\omega(t) = \mathbf{1}[t, t + \tau]$. This windowing mechanism enables the system to capture localized dynamic behaviors by restricting the support of the basis to a finite temporal region. When $\phi_n(t, s)$ is instantiated as a set of trigonometric functions, the resulting operator reduces to the Short-Time Fourier Transform (STFT) [9], which is widely used for time–frequency analysis and as an embedding technique in audio processing.

Hippo-LegT is a notable variant that employs Legendre polynomials as the orthogonal basis $\phi_n(t, s)$. Recent work [15] has demonstrated that LegT is particularly effective in capturing periodic structures in time-series data. Motivated by these findings, we adopt LegT as one of the structured transition matrices in our experiments. The corresponding transition matrix $\mathbf{A}$ is defined as:

$$\mathbf{A}_{nk}^{(N)} = - \begin{cases} (2n+1)^{\frac{1}{2}}(2k+1)^{\frac{1}{2}} & n < k, k \text{ odd} \\ 0 & \text{else} \\ (2n+1)^{\frac{1}{2}}(2k+1)^{\frac{1}{2}} & n > k, n \text{ odd} \end{cases} \quad (4)$$

$$\mathbf{A} = \mathbf{A}^{(D)} - \text{rank}(2), \quad \mathbf{A}^{(D)} := \text{eig}(\mathbf{A}^{(N)})$$

iii) **Laplace Transform**. can be realized with:

$$\omega(t, s) = e^{-\sigma t}, \quad p_n(t, s) = e^{-its}.$$

This formulation introduces an exponential decay term via the measurement $\omega(t, s)$, which serves as a temporal weighting mechanism that prioritizes recent inputs while gradually discounting older information. Such decay measures are particularly beneficial for modeling long-term dependencies, as they enable the system to retain and integrate distant past information in a stable manner.

S4-LegS² is a Legendre-based variant that incorporates this exponential decay structure into the state-space formulation for long-range dynamics. Motivated by these advantages, we adopt S4-LegS as the second structured transition matrix in our design. The resulting matrix $\mathbf{A}$ is defined as:

²We adopt the LegS initialization proposed in S4 [12], which differs from the original HiPPO-LegS. According to HTTYH [13], the transformation is given by $\text{S4-LegS}(\mathbf{A}, \mathbf{B}) = \text{HiPPO-LegS}(\frac{1}{t}\mathbf{A}, \frac{1}{t}\mathbf{B})$ [13].

$$A_{nk}^{(N)} = - \begin{cases} (n + \frac{1}{2})^{1/2} (k + \frac{1}{2})^{1/2} & n > k \\ \frac{1}{2} & n = k \\ (n + \frac{1}{2})^{1/2} (k + \frac{1}{2})^{1/2} & n < k \end{cases} \quad (5)$$

$$A = A^{(D)} - \text{rank}(1), \quad A^{(D)} := \text{eig}(A^{(N)})$$

iv) **Wavelet Transform** can be realized with:

$$\omega(t, s) = \mathbf{1}_{[0,1]}, \quad p_n(t, s) = |m|^{-\frac{1}{2}} \phi_n\left(\frac{t - \tau}{m}\right).$$

In this setting, the basis functions ϕ_n are both scaled by a factor m and shifted by a parameter τ , allowing the system to model temporal dynamics at multiple resolutions. This multi-scale structure enables the representation of both short-term transients and long-term patterns within a unified framework. Such flexibility makes the wavelet-based approach particularly effective for capturing non-stationary characteristics often observed in real-world time series, such as abrupt changes and scale-varying structures.

Notably, **Attraos** [15] can be interpreted as a state-space model instantiation under this wavelet-based state transition to model multi-scale attractors. Moreover, this framework is kernel-agnostic, and it can be seamlessly extended by incorporating a loop on top of the previously discussed SSM kernel types.

v) **Selective SSMs as Adaptive Spectral Operator.** The recent SSM variant Mamba [10] introduces a time-varying (data-dependent) selection mechanism, computing the B and C matrices via linear projections and initializing A to $\text{Diag}(-1, -2, \dots)$, instead of the Hippo initialization [1, 11] in early SSMs:

$$\begin{aligned} \bar{A}_t^{\text{all}} &= \text{Broadcast}_D(\text{Diag}(-1, -2, \dots)) \in \mathbb{R}^{D \times N \times N}, \\ B_t, C_t &= \text{Linear}_{BC}(u) \in \mathbb{R}^N. \end{aligned} \quad (6)$$

Moreover, for real-world time-series applications, it is essential to convert the continuous-time parameters $(\Delta A B)$ into their discrete-time counterparts $(\bar{A}, \bar{B})$. In Mamba, this conversion leverages a combination of the zero-order hold method—obtained by solving Lemma 2.1—and the forward Euler method [11], yielding a more concise and efficient parameterization.

$$\begin{aligned} (\text{ZOH}) : \bar{A}_t &= \exp(\Delta_t A_t) \in \mathbb{R}^{D \times N \times N}, \\ (\text{Forward Euler}) : \bar{B}_t &= \Delta B_t \in \mathbb{R}^{D \times N}, \\ \Delta_t &= \text{softplus}(\text{Linear}_\Delta(u)) \in \mathbb{R}^D. \end{aligned} \quad (7)$$

From a spectral-transform perspective, Mamba can be interpreted as an adaptive spectral method: it uses gradient descent to learn both the projection and output mappings, enabling data-dependent optimization of the spectral representation. In addition, the learnable discrete step size Δ endows the SSM with a selective attention mechanism, enabling it to prioritize important temporal information—analogueous to the behavior of attention-based models. We visualize this process in Fig. 2 for understanding. Furthermore, by computing the outer product of A and B , the model expands the dimensionality of the state transition and input matrices, allowing channel-mixing interactions across input dimensions. This stands in contrast to models like S4, where each input channel is modeled independently, limiting inter-channel dependencies.

vi) **Beyond Mamba.** Building on Mamba, Mamba2 [8] introduces the multi-head computation mechanism commonly used in Transformer architectures. Within each head, the state transition matrix is simplified to a scalar, which can be interpreted as a diagonal matrix with identical entries. In fact, **Mamba2**³ exhibits a reduced expressive capacity in terms of state transitions:

$$\begin{aligned} \bar{A}_t^{\text{all}} &= \text{Concat} \sum_i^{\text{heads}} \text{Diag}(\alpha_t^i, \alpha_t^i, \dots) \in \mathbb{R}^{\text{heads} \times 1 \times 1}, \\ \alpha_t^i &= \text{sigmoid}(\text{Linear}_a^i(x)) \in \mathbb{R}, \\ B_t, C_t &= \text{Linear}_{BC}(u) \in \mathbb{R}^N. \end{aligned} \quad (8)$$

Empirically, this modification resembles the transition from **LegS** to **LegT** (From Laplace transform to the Gabor transform). Specifically, the negative-valued diagonals in Mamba1 (e.g., $-1, -2, -3$) correspond to the time-decaying measures used in LegS, while the uniform diagonal entries in Mamba2 resemble the constant, non-decaying measure adopted in LegT. This change enhances the uniformity of information propagation across time. At the same time, the design of **Mamba2** also signifies a unification between state-space models and linear attention mechanisms. In this formulation, the input projection matrix B can be interpreted as the **key**, while the output projection matrix C plays the role of the **query**.

However, in language modeling, where next-token prediction is essentially a classification task, Mamba simplifies A from a dense matrix to a diagonal one to improve efficiency. For continuous-valued tasks like TSF, it remains unclear whether such simplification is justified. This uncertainty serves as one of the key motivations behind this work. Inspired by Comba [16], the latest variant of Mamba, we also construct a fully trainable structured transition matrix as our fourth experimental configuration. This matrix is a combination of scalar and low-rank components (SPLR), both generated through linear layers. As a result, the overall A matrix becomes a learnable symmetric matrix, no longer constrained to be diagonal. Moreover, to control for confounding factors, we fix the number of heads to one, which means no multi-head decomposition is applied. In Table 1, we summarize our proposed structured transition matrices used in our experiments.

$$\begin{aligned} \bar{A}_t^{\text{all}} &= \text{Concat} \sum_i^{\text{heads}} (\alpha_t^i - b_t^i b_t^{i\top}) \in \mathbb{R}^{\text{heads} \times N \times N}, \\ \alpha_t^i &= \text{sigmoid}(\text{Linear}_a(x)) \in \mathbb{R}, \quad b_t^i = \text{Linear}_b(x) \in \mathbb{R}^{N/\text{heads}} \\ B_t, C_t &= \text{Linear}_{BC}(u) \in \mathbb{R}^N. \end{aligned} \quad (9)$$

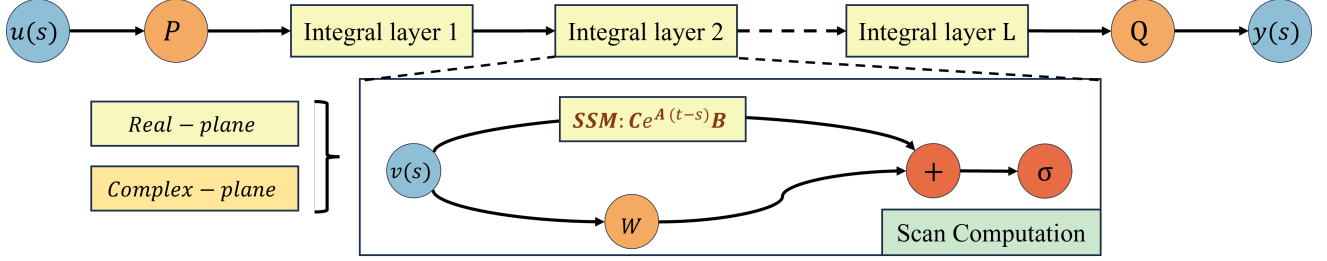
4 Time-SSM: SSM Kernels for TSF

Problem Definition. In this paper, we formulate time-series forecasting as an *operator learning* problem. Given an input sequence $u(s)$, we employ a state-space model (SSM) to approximate

³In practice, Mamba2 employs a more fine-grained initialization scheme for its state transition α to ensure its values remain $(0, 1)$. So we present an equivalent formulation using a sigmoid function for clarity.

Table 1: Parameterization choices for structured SSM matrices in the experiments. LegT and LegS stabilize training gradients by conjugate-diagonalizing the original normal-plus-low-rank (NPLR) form into complex numbers.

Time-varying SSMs	Structure of A	Structure of B	Structure of C	Time-varying factor
Mamba	$\mathbb{R}(D, N) \leftarrow \text{real diagonal}$	$\mathbb{R}(B, L, N) : \text{Linear}_B(x)$	$\mathbb{R}(B, L, N) : \text{Linear}_C(x)$	Δ, B, C
Mamba2	$\mathbb{R}(D, 1) \leftarrow \text{scalar}$	$\mathbb{R}(B, L, N) : \text{Linear}_B(x)$	$\mathbb{R}(B, L, N) : \text{Linear}_C(x)$	Δ, B, C
LegS/LegT	$\mathbb{R}(D, N, N) \leftarrow \text{full matrix}$	$\mathbb{R}(B, L, N) : \text{Linear}_B(x)$	$\mathbb{R}(B, L, N) : \text{Linear}_R(x)$	Δ, B, C
LegS/LegT Realization	$\mathbb{C}(D, N) \leftarrow \text{complex diagonal}$	$\mathbb{C}(B, L, N) : V^{-1} \text{Linear}_B(x)$	$\mathbb{C}(B, L, N) : V \text{Linear}_C(x)$	Δ, B, C
Comba	$\mathbb{R}(D, N, N) \leftarrow \text{SPLR}$	$\mathbb{R}(D, N) \leftarrow \text{parameter}$	$\mathbb{R}(D, N) \leftarrow \text{parameter}$	Δ, B, C

**Figure 3: Model pipeline in this paper, which is an adaptive neural operator with kernel $C \exp(A(t-s))B$ from input $u(s)$ to $y(s)$.**

a neural operator $\mathcal{K}(t, s)$ that learns to project the data into an appropriate spectral domain, where the complex mapping to the future time series $y(s)$ can be effectively learned.

As shown in Fig. 3, we adhere to TSF standards [30] by applying normalization and de-normalization techniques to the input and output, addressing the distribution shift issues [20]. Furthermore, following the standard setting in neural operators [17], we employ linear embedding operations P on time series patches [22] to obtain a vectorized representation of the input function $u(s)$ and use linear mapping Q back to the numerical observation domain $y(s)$.

In the following, we introduce the variants of SSM kernels employed in our experiments.

SSM Kernel without Hippo Initialization. We construct three versions of time-varying SSM kernels based on Equations (6-9), corresponding to three types of state transition matrices: diagonal, scalar, and structured scalar plus low rank. All SSM kernel parameters are initialized from a normal distribution and updated via gradient-based backpropagation during training.

SSM Kernel with Hippo Initialization. The essence of state-space models (SSMs) lies in computing the *Krylov kernel* induced by the transition matrix A , which encodes the dynamic behavior of the system. To this end, we investigate the use of *structured initializations* for A while preserving the time-varying nature of the model. Specifically, we construct time-varying variants of **Hippo-LegT** and **S4-LegS**, aiming to combine the benefits of inductive structure with dynamic adaptability.

However, this design leads to an expanded state transition tensor of dimension $\tilde{A} = \Delta A \in \mathbb{R}^{B \times L \times D \times N \times N}$, and in practice, we observe that this high-dimensional parameterization introduces *gradient explosion* issues during training, as the eigen-value of $\tilde{A}$ becomes increasingly difficult to constrain in $(0, 1)$.

To mitigate this, we adopt the approach introduced in S4 [12] and apply a *conjugate diagonalization* of the real-valued transition matrices, mapping them onto the complex plane. As shown in Lemma 4.1, this operation constitutes a coordinate transformation that preserves the underlying system dynamics.

LEMMA 4.1. *For the n -dimensional SSM(A, B, C), any transformation defined by a nonsingular matrix T will generate a transformed SSM($\hat{A}, \hat{B}, \hat{C}$): [Conjugate diagonalizing]*

$$\hat{A} = T^{-1}AT, \quad \hat{B} = T^{-1}B, \quad \hat{C} = CT,$$

where the dynamic matrix $\hat{A}$ has the same characteristic polynomial and eigenvalues (same system dynamics) as A , but its eigenvectors (coordinate space) are different.

Specifically, we diagonalize the matrix A in the complex domain using the unitary matrix V , such that $A = V^{-1}AV$ where $\Lambda = A^{(D)} = \text{eig}(A^{(N)})$. This ensures that A is unitarily equivalent to $A^{(D)}$. We then integrate the unitary matrix $V \in \mathbb{C}^{N \times N}$ with the time-varying BC generated by the linear layer, thereby enabling the model to exhibit compatible dynamics. It is important to note that the unitary matrix satisfies unitary matrix $V^T = V^{-1}$.

$$B = V^T \text{Linear}_B(u), \quad C = V \text{Linear}_C(u). \quad (10)$$

From this perspective, Mamba can be viewed as a specially structured SSM that discards the complex plane values, i.e., $\Im(ABC) = 0$. In language modeling tasks, it has been shown that discarding complex plane information typically requires doubling the real-valued dimensions to compensate for the loss [24].

5 Analysis: How to Train Your Mamba in TSF?

In this section, all experiments are implemented using the Time Series Library⁴ and conducted on NVIDIA A6000-48G GPUs. We adopt the Adam optimizer and perform a grid search over key

⁴<https://github.com/thuml/Time-Series-Library>

hyperparameters: patch length is selected from $\{8, 16\}$, and the number of model layers from $\{1, 2\}$. The hidden dimension is set to 256, the state space dimension to 64, and the input sequence length to 96. The standard SSM kernels are based on the official codebase⁵, and the fully trainable structured SSM kernel (Equation 9) is based on the open-source Comba implementation⁶.

We aim to analyze the following questions gradually:

- **RQ1:** Is the Mamba architecture truly effective in TSF?
- **RQ2:** What kind of state transition matrix is best?
- **RQ3:** Is specialized initialization necessary?
- **RQ4:** What are the other properties of SSMs in TSF?

5.1 Answer for RQ1: Mamba is not Optimal

As shown in Table 2, we remove auxiliary components from Mamba, such as the short convolution, leaving only the core SSM kernel for experimentation. Structurally, we find that the Mamba kernel consistently outperforms the full Mamba architecture and original S4 [12], which utilizes fixed Hippo matrices. We argue that:

The truly effective component of Mamba lies in its *time-varying selection mechanism*—that is, the data-dependent SSM kernel. In contrast, auxiliary modules such as short convolutions or gating layers often introduce task-agnostic inductive biases that may lead to overfitting in time-series forecasting tasks [35]. Empirically, we find that retaining only the SSM kernel results in more robust and generalizable performance across diverse datasets.

As shown in Table 3, we further investigate the roles of other Mamba components. Specifically, we eliminate the adaptive projection of B and instead initialize it with S4-LegS, which indirectly equips B with time-varying characteristics through the time-varying factor Δ , i.e., $\bar{B}_t = \Delta_t B_t$. We argue that: Initializing B using a learnable linear layer is not universally beneficial; in some cases, leveraging inherent system dynamics offers a more stable and effective initialization strategy. Moreover, residual connections are not strictly necessary, particularly in shallow temporal forecasting models with only two layers, and may even contribute to overfitting. For non-stationary time-series data, we observe that lightweight architectures often outperform deeper or more complex alternatives, likely due to their reduced capacity to memorize noise and their improved generalization.

5.2 Answer for RQ2: Structured Matrix is Better

As shown in Table 4, we replace the Mamba kernel with various SSM kernel variants introduced in Section 4 to evaluate which formulation yields the best performance in time-series forecasting.

Our results suggest that the structure of Mamba’s state transition matrix is not optimal for time-series forecasting. In particular, **Comba**, which employs a structured full transition matrix, demonstrates stronger representational capacity than Mamba’s diagonal formulation, leading to superior forecasting performance. Furthermore, our time-varying variant of **LegT** utilizes a complex-valued diagonal transition matrix, which—by operating in the complex domain—effectively doubles the degrees of freedom compared to Mamba’s real-valued diagonal matrix. This theoretical advantage translates into consistently improved empirical performance.

⁵<https://github.com/state-spaces>

⁶<https://github.com/AwesomeSeq/Comba-triton>

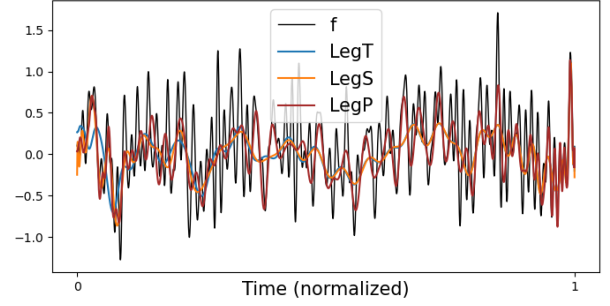


Figure 4: Long-range function approximation with different SSM basis. LegP is the wavelet version in Section 3.

Notably, although **Mamba2** represents a newer generation of Mamba variants, its use of a scalar transition matrix inherently limits its expressive power. From a neural operator perspective, scalar transitions lack the richness of diagonal or structured transitions, making them less suited for modeling complex temporal dependencies in forecasting scenarios.

5.3 Answer for RQ3: Initialization is Important

Similarly, Table 4 reveals that the **LegS**-based kernels consistently underperform their **LegT** counterparts, both in time-invariant and time-varying settings. This suggests that the decay-based initialization may not align well with the dynamics of forecasting tasks. In addition, we conduct a controlled experiment in which the transition matrix A in the Mamba kernel is replaced with a fully learnable diagonal matrix initialized from a standard normal distribution. We refer to this variant as Full-Select. As shown in the table, this modification results in a significant drop in forecasting accuracy. Together, these findings highlight the critical role of initialization in SSM kernels, particularly for the transition matrix A . Carefully designed parameter initializations, such as hand-crafted diagonals or theoretically grounded bases (e.g., LegT, LegS), provide meaningful inductive biases that can be essential for stable and effective learning in time-series forecasting [1].

5.4 Answer for RQ4: Further Analysis

In this section, we further explore several additional and intriguing properties of SSMs in the context of TSF tasks.

Representation Ability. To further support our conclusion that specific state transition dynamics exhibit distinct levels of expressiveness, we conduct a set of controlled synthetic experiments. We utilize a 64-dimensional (same as in our experiments) fixed Hippo matrix to reconstruct the input function, in order to explore the representational capacity of different SSM kernels. The input function is randomly sampled from a continuous-time band-limited white noise process with a length of 10^6 , a sampling step size of 10^{-4} , and a signal band limit of 1Hz. We further extend the LegT-based SSM kernel using the wavelet-inspired formulation described in Section 3, which performs piecewise polynomial approximation. Specifically, we apply a segment-wise decomposition with a scale of

Table 2: Average results of long-term forecasting with an input length of 96 and prediction horizons of {96, 192, 336, 720}. The best performance is in Red, and the second best is in Blue.

Model	Mamba kernel		Mamba4TS		S-Mamba		RWKV-TS		Koopa		InvTrm		PatchTST		DLinear		S4	
Metric	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTh1	0.430	0.426	0.444	0.438	0.459	0.453	0.454	0.446	0.450	0.443	0.463	0.454	0.434	0.435	0.462	0.458	0.591	0.533
ETTh2	0.374	0.399	0.386	0.410	0.381	0.407	0.375	0.402	0.397	0.417	0.383	0.407	0.380	0.406	0.564	0.520	0.406	0.439
ETTm1	0.386	0.396	0.396	0.406	0.399	0.407	0.391	0.403	0.395	0.403	0.407	0.412	0.403	0.398	0.403	0.406	0.419	0.423
ETTm2	0.283	0.328	0.299	0.343	0.289	0.333	0.285	0.330	0.281	0.326	0.291	0.335	0.283	0.329	0.345	0.396	0.314	0.361
Exchange	0.352	0.398	0.364	0.405	0.364	0.407	0.406	0.439	0.390	0.424	0.366	0.416	0.383	0.416	0.346	0.416	0.534	0.529
Crypto	0.192	0.160	0.193	0.162	0.198	0.163	0.190	0.159	0.199	0.165	0.196	0.164	0.192	0.161	0.201	0.176	0.221	0.194
Air-convection	0.459	0.332	0.471	0.343	0.484	0.352	0.464	0.336	0.463	0.337	0.493	0.363	0.483	0.354	0.459	0.341	0.476	0.361
Weather	0.252	0.276	0.258	0.280	0.252	0.277	0.256	0.280	0.247	0.273	0.260	0.280	0.258	0.280	0.267	0.319	0.264	0.297
1 st /2 st Count	8	8	0	1	1	0	3	2	3	2	0	0	3	2	2	1	0	0

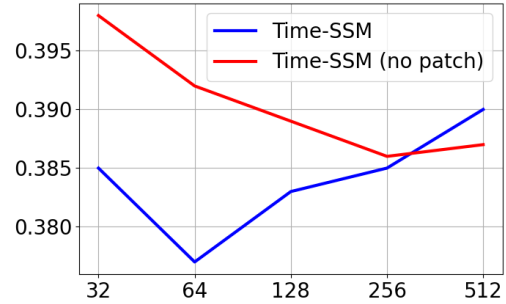
Table 3: Ablation Study on the Time-Varying Behavior of the B Matrix and the Residual Connection (W Matrix).

Model		Mamba kernel		w/o B_t		w/o W	
Metric		MSE	MAE	MSE	MAE	MSE	MAE
ETTh1	96	0.377	0.394	0.382	0.401	0.380	0.400
	192	0.423	0.424	0.438	0.436	0.432	0.429
	336	0.466	0.437	0.483	0.452	0.476	0.452
	720	0.452	0.448	0.510	0.489	0.497	0.471
	AVG	0.430	0.426	0.453	0.445	0.446	0.438
ETTh2	96	0.290	0.341	0.298	0.349	0.288	0.339
	192	0.368	0.387	0.382	0.403	0.371	0.390
	336	0.416	0.430	0.420	0.427	0.412	0.425
	720	0.424	0.439	0.422	0.441	0.418	0.438
	AVG	0.374	0.399	0.381	0.405	0.372	0.398
Air	96	0.290	0.341	0.292	0.343	0.286	0.338
	192	0.368	0.387	0.366	0.386	0.372	0.391
	336	0.416	0.430	0.411	0.423	0.419	0.434
	720	0.424	0.439	0.419	0.434	0.432	0.447
	AVG	0.374	0.399	0.372	0.396	0.377	0.403

3 and refer to this variant as LegP. From Figure 4, different SSM kernels exhibit clear differences in approximation capability. Among them, LegP demonstrates the strongest expressiveness due to its multi-scale structure introduced by piecewise polynomial decomposition. Interestingly, the approximation ability of LegS appears to surpass that of LegT, which contrasts with our empirical forecasting results. While one explanation may be that short-range temporal dependencies in time-series data do not benefit from the decaying structure of LegS, another possible reason is that LegT provides more robust and stable predictions in practice, which is critical for generalization in forecasting tasks.

LEMMA 5.1. *Let $\mathcal{F}$ be a nonlinear dynamical system governed by $\dot{x} = f(x(t), t)$ with smooth nonlinear dynamics $f : \mathbb{R}^m \rightarrow \mathbb{R}^m$. Suppose $\mathcal{F}$ is partitioned into local patches $\{P_i\}$, then all points $x \in P_i$ shares the same linearized dynamics $\dot{x} \approx A_i x + b_i$, $A_i = \left. \frac{\partial f}{\partial x}(x) \right|_{x \in P_i}$.*

Patch operation. Theoretically, the expressive capacity of SSMs is governed by the degree of the underlying polynomial approximation [11, 15], which corresponds to the model’s state size. In general,

**Figure 5: The relationship between the expressive capacity of the SSM kernel and the state size on the ETTh2 dataset.**

an order of 256 is required to achieve strong universal approximation capabilities. However, in time-series forecasting tasks, it is common practice to apply patching operations that divide the input sequence into shorter segments. Consistent with observations in **Attras** [15], we find that the patching operation linearly reduces the polynomial order required for effective approximation. Lemma 5.1 suggests that patching serves as a form of local approximation that significantly alleviates the burden on the kernel’s state size.

Implicit Autoregression Forecasting. SpaceTime [36] proposed a closed-loop control prediction method using input time series $u_{[1:L]}$ to construct a dynamical system and generate predictions $y_{[L+1:L+H]}$ through autoregressive modeling. However, this approach is not suitable for time-varying systems with time varying matrices (**ABC**). To address this, we introduced an implicit autoregressive method with zero padding: $y \in \mathbb{R}/\mathbb{C}^{L+H} = \text{Time-SSM} (u \in \mathbb{R}/\mathbb{C}^{L+O})$. The final prediction is obtained by truncating the last H steps of y . Experimental results (Table 5) show decreased performance for all initialization methods under this configuration, indicating redundancy of the additional autoregressive generation. Surprisingly, in longer sequences, the LegS-based variant outperforms LegT, contrary to Table 4 findings, supporting the original benefit of LegS for long-term sequence modeling. Moreover, selecting the **BC** matrix from the last time step for autoregressive generation leads to

Table 4: Ablation study of various SSM kernel ABC parameterization. Our-XX refers to our proposed time-varying variants of the LegT and LegS kernels, while Comba denotes a fully data-dependent structured SSM kernel. Red/Blue denote best/second performance per row.

SSMs		Mamba		Our LegS		Our LegT		Mamba2		Comba		S4-LegS		Hippo-LegT		Full-Select	
Metric		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTh1	96	0.377	0.394	0.382	0.398	0.374	0.391	0.380	0.397	0.369	0.388	0.391	0.405	0.388	0.403	0.382	0.399
	192	0.423	0.424	0.429	0.427	0.420	0.421	0.426	0.429	0.416	0.417	0.438	0.436	0.434	0.433	0.435	0.429
	336	0.466	0.437	0.471	0.450	0.464	0.448	0.468	0.445	0.467	0.440	0.475	0.451	0.470	0.445	0.482	0.452
	720	0.452	0.448	0.474	0.471	0.455	0.452	0.459	0.456	0.456	0.451	0.488	0.483	0.484	0.477	0.483	0.476
	AVG	0.430	0.426	0.439	0.437	0.428	0.428	0.433	0.432	0.427	0.424	0.448	0.444	0.444	0.440	0.446	0.439
ETTm2	96	0.176	0.260	0.179	0.265	0.172	0.260	0.178	0.262	0.169	0.257	0.188	0.274	0.184	0.273	0.180	0.265
	192	0.246	0.305	0.246	0.308	0.247	0.302	0.248	0.308	0.243	0.300	0.264	0.321	0.262	0.318	0.247	0.309
	336	0.305	0.344	0.310	0.348	0.300	0.340	0.309	0.349	0.307	0.346	0.321	0.358	0.317	0.355	0.312	0.351
	720	0.406	0.405	0.411	0.410	0.405	0.403	0.404	0.401	0.402	0.399	0.431	0.436	0.430	0.428	0.412	0.410
	AVG	0.283	0.329	0.287	0.333	0.283	0.328	0.283	0.328	0.281	0.326	0.301	0.347	0.298	0.343	0.288	0.334
Weather	96	0.167	0.212	0.175	0.218	0.178	0.221	0.165	0.210	0.169	0.214	0.189	0.228	0.184	0.222	0.185	0.225
	192	0.217	0.255	0.222	0.253	0.223	0.254	0.216	0.253	0.214	0.248	0.238	0.269	0.236	0.263	0.230	0.262
	336	0.274	0.294	0.285	0.303	0.271	0.295	0.276	0.298	0.273	0.299	0.291	0.310	0.294	0.311	0.285	0.302
	720	0.351	0.345	0.354	0.346	0.350	0.342	0.349	0.344	0.348	0.339	0.366	0.358	0.362	0.355	0.360	0.351
	AVG	0.252	0.277	0.259	0.280	0.256	0.278	0.252	0.276	0.251	0.275	0.271	0.291	0.269	0.288	0.265	0.285
1 st /2 nd Count		8	6	0	1	8	7	5	4	12	13	0	0	0	0	0	0

Table 5: Forecasting results in AR process with various prediction horizons. ∞ means gradient explosions.

SSMs		Mamba		Our LegS		Our LegT		Wavelet-LegT		Full-Select		S4-LegS	
Metric		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTm2	96	0.184	0.272	0.189	0.278	0.188	0.275	0.181	0.270	0.192	0.276	0.198	0.296
	192	0.251	0.313	0.255	0.319	0.254	0.316	0.251	0.312	0.256	0.316	0.264	0.327
	336	0.320	0.360	0.320	0.359	0.324	0.362	0.324	0.360	0.317	0.354	0.337	0.368
	720	0.420	0.417	0.417	0.413	0.418	0.415	0.420	0.417	0.423	0.413	∞	∞
	AVG	0.294	0.341	0.295	0.342	0.296	0.342	0.294	0.340	0.297	0.340	∞	∞
Weather	96	0.182	0.223	0.190	0.229	0.194	0.234	0.186	0.228	0.190	0.229	0.186	0.227
	192	0.227	0.260	0.234	0.266	0.242	0.273	0.236	0.269	0.239	0.269	0.236	0.270
	336	0.282	0.300	0.289	0.304	0.298	0.312	0.284	0.299	0.295	0.308	0.315	0.330
	720	0.361	0.351	0.362	0.351	0.375	0.362	0.361	0.350	0.366	0.354	∞	∞
	AVG	0.263	0.284	0.269	0.288	0.277	0.295	0.267	0.287	0.273	0.290	∞	∞
1 st /2 nd Count		9	8	5	4	2	1	7	6	1	3	1	1

gradient explosions in time-varying environments due to the lack of dynamic constraints.

6 Related Work

Neural Operator. Given two functions $u(s)$ and $y(t)$, the operator is a map $\mathcal{K}$ such that $\mathcal{K}u = y$ in Sobolev spaces $\mathcal{H}^{b,d}$. Typically, we choose $b > 0$ and $d = 2$ to facilitate the definition of projections w.r.t. measures μ in a Hilbert space structure. We take the operator $\mathcal{K} : u \mapsto y$ as an integral operator in interval D with the kernel $K(t, s)$, referring to Figure 1(a).

$$\mathcal{K}u(s) = \int_D K(t, s)u(s)ds, \quad (11)$$

$$K(t, s) = \mathbb{T} \circ \hat{K}(t, s) \circ \mathbb{T}^{-1}. \quad (12)$$

Learning the complete kernel $K(t, s)$ would essentially solve the operator map problem. However, it is not necessarily a numerically feasible solution. An overarching approach is to obtain a compact representation $\hat{K}$ of the complete sparse kernel K , which necessitates one or a sequence of suitable domains along with projection tools $\mathbb{T}$ to these domains as Eq. 12, referring to Figure 1(b).

Efficient Computation for SSM. While SSMs provide an efficient pretraining time complexity of $\mathcal{O}(LD^2)$, they tend to be slower than softmax attention (Transformer), which has a complexity of $\mathcal{O}(L^2D)$, especially on shorter sequences. This discrepancy arises primarily because current hardware is highly optimized for matmul operations, limiting the effectiveness of linear recurrence and requiring additional training optimizations. To this end, models such as S5 [27] and Mamba [10] employ the Blelloch scan algorithm [4] to cache intermediate states, enabling efficient recursive computation

by first processing the even-indexed positions of the sequence, followed by the odd-indexed positions. More recent methods, inspired by FlashAttention [7], Mamba2 [8] introduce inter-chunk recurrence combined with intra-chunk parallelism, optimizing matrix compute throughput and Hardware utilization rate.

Deep Time Series Forecasting Architecture. The development of time-series forecasting models has largely mirrored the broader trajectory of deep learning. Early research focused on adopting Transformer-based [5, 22, 37] or convolutional neural network (CNN) [29, 33] architectures as model backbones, with some efforts incorporating frequency-domain representations [38] and non-stationary characteristics [20] into architectural design. Meanwhile, other works have recognized the inherently lightweight nature of forecasting tasks and have shifted toward simpler linear mappings [35] or lightweight convolutional structures [19]. More recently, inspired by the success of state-space models (SSMs) in computer vision and natural language processing, SSMs have emerged as promising candidates for time-series forecasting as well.

7 Conclusion

State-space models have become strong candidates for deep time-series forecasting. However, there is still limited understanding of why they are effective and which design choices lead to better performance. This paper presents a comprehensive analysis of SSMs in forecasting tasks. We demonstrate that the current Mamba architecture is not optimal and highlight the importance of transition dynamics and initialization strategies. These findings provide useful insights for future research. In future work, we plan to explore the role of SSMs in large-scale time-series foundation models [18].

Acknowledgement

This work is supported by the Guangdong Basic and Applied Basic Research Foundation (No. 2025A1515011994), the National Natural Science Foundation of China (No. 62402414), Guangzhou Municipal Science and Technology Project (No. 2023A03J0011), the Guangzhou Industrial Information and Intelligent Key Laboratory Project (No. 2024A03J0628), and Guangdong Provincial Key Lab of Integrated Communication, Sensing and Computation for Ubiquitous Internet of Things (No. 2023B1212010007).

References

- [1] Ido Amos, Jonathan Berant, and Ankit Gupta. 2023. Never Train from Scratch: Fair Comparison of Long-Sequence Models Requires Data-Driven Priors. *arXiv preprint arXiv:2310.02980* (2023).
- [2] Md Atik Ahamed and Qiang Cheng. 2024. TimeMachine: A Time Series is Worth 4 Mambas for Long-term Forecasting. *arXiv e-prints* (2024), arXiv-2403.
- [3] Ali Behrouz, Michele Santacatterina, and Ramin Zabih. 2024. MambaMixer: Efficient Selective State Space Models with Dual Token and Channel Selection. *arXiv preprint arXiv:2403.19888* (2024).
- [4] Guy E Blelloch. 1990. Prefix sums and their applications. (1990).
- [5] Peng Chen, Yingying Zhang, Yunyao Cheng, Yang Shu, Yihang Wang, Qingsong Wen, Bin Yang, and Chenjuan Guo. 2024. Pathformer: Multi-scale transformers with Adaptive Pathways for Time Series Forecasting. *arXiv preprint arXiv:2402.05956* (2024).
- [6] Tri Dao. 2023. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691* (2023).
- [7] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems* 35 (2022), 16344–16359.
- [8] Tri Dao and Albert Gu. 2024. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality. *arXiv preprint arXiv:2405.21060* (2024).
- [9] Lutfiye Durak and Orhan Arikan. 2003. Short-time Fourier transform: two fundamental properties and an optimal implementation. *IEEE Transactions on Signal Processing* 51, 5 (2003), 1231–1242.
- [10] Albert Gu and Tri Dao. 2023. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752* (2023).
- [11] Albert Gu, Tri Dao, Stefano Ermon, Atri Rudra, and Christopher Ré. 2020. Hippo: Recurrent memory with optimal polynomial projections. *Advances in neural information processing systems* 33 (2020), 1474–1487.
- [12] Albert Gu, Karan Goel, and Christopher Ré. 2021. Efficiently modeling long sequences with structured state spaces. *arXiv preprint arXiv:2111.00396* (2021).
- [13] Albert Gu, Isys Johnson, Aman Timalina, Atri Rudra, and Christopher Ré. 2022. How to train your hippo: State space models with generalized orthogonal basis projections. *arXiv preprint arXiv:2206.12037* (2022).
- [14] Ankit Gupta, Albert Gu, and Jonathan Berant. 2022. Diagonal state spaces are as effective as structured state spaces. *Advances in Neural Information Processing Systems* 35 (2022), 22982–22994.
- [15] Jiaxi Hu, Yuehong Hu, Wei Chen, Ming Jin, Shirui Pan, Qingsong Wen, and Yuxuan Liang. 2024. Attractor Memory for Long-Term Time Series Forecasting: A Chaos Perspective. *arXiv preprint arXiv:2402.11463* (2024).
- [16] Jiaxi Hu, Yongqi Pan, Jusen Du, Disen Lan, Xiaqiang Tang, Qingsong Wen, Yuxuan Liang, and Weigao Sun. 2025. Comba: Improving Nonlinear RNNs with Closed-loop Control. *arXiv preprint arXiv:2506.02475* (2025).
- [17] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. 2020. Fourier neural operator for parametric partial differential equations. *arXiv preprint arXiv:2010.08895* (2020).
- [18] Yuxuan Liang, Haomin Wen, Yuqi Nie, Yushan Jiang, Ming Jin, Dongjin Song, Shirui Pan, and Qingsong Wen. 2024. Foundation models for time series analysis: A tutorial and survey. *arXiv preprint arXiv:2403.14735* (2024).
- [19] Shengsheng Lin, Weiwei Lin, Wentai Wu, Haojun Chen, and Junjie Yang. 2024. SparseTSF: Modeling Long-term Time Series Forecasting with 1k Parameters. *arXiv preprint arXiv:2405.00946* (2024).
- [20] Yong Liu, Haixu Wu, Jianmin Wang, and Mingsheng Long. 2022. Non-stationary transformers: Exploring the stationarity in time series forecasting. *Advances in Neural Information Processing Systems* 35 (2022), 9881–9893.
- [21] Stefano Markidis, Steven Wei Der Chien, Erwin Laure, Ivy Bo Peng, and Jeffrey S Vetter. 2018. Nvidia tensor core programmability, performance & precision. In *2018 IEEE international parallel and distributed processing symposium workshops (IPDPSW)*. IEEE, 522–531.
- [22] Yuqi Nie, Nam H Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. 2022. A time series is worth 64 words: Long-term forecasting with transformers. *arXiv preprint arXiv:2211.14730* (2022).
- [23] Badri N Patro and Vijay S Agneeswaran. 2024. SIMBA: Simplified Mamba-Based Architecture for Vision and Multivariate Time series. *arXiv preprint arXiv:2403.15360* (2024).
- [24] Zhen Qin, Songlin Yang, Weixuan Sun, Xuyang Shen, Dong Li, Weigao Sun, and Yiran Zhong. 2024. Hgrn2: Gated linear rnns with state expansion. *arXiv preprint arXiv:2404.07904* (2024).
- [25] Ricardo Buitrago Ruiz and Albert Gu. 2025. Understanding and Improving Length Generalization in Recurrent Models. *arXiv preprint arXiv:2507.02782* (2025).
- [26] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. 2024. Flashattention-3: Fast and accurate attention with asynchrony and low-precision. *Advances in Neural Information Processing Systems* 37 (2024), 68658–68685.
- [27] Jimmy TH Smith, Andrew Warrington, Scott W Linderman, and else. 2022. Simplified state space layers for sequence modeling. *arXiv preprint arXiv:2208.04933* (2022).
- [28] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [29] Huiqiang Wang, Jian Peng, Feihu Huang, Jince Wang, Junhui Chen, and Yifei Xiao. 2022. Micn: Multi-scale local and global context modeling for long-term series forecasting. In *The Eleventh International Conference on Learning Representations*.
- [30] Yuxuan Wang, Haixu Wu, Jiaxiang Dong, Yong Liu, Mingsheng Long, and Jianmin Wang. 2024. Deep time series models: A comprehensive survey and benchmark. *arXiv preprint arXiv:2407.13278* (2024).
- [31] Zihan Wang, Fanheng Kong, Shi Feng, Ming Wang, Han Zhao, Daling Wang, and Yifei Zhang. 2024. Is Mamba Effective for Time Series Forecasting? *arXiv preprint arXiv:2403.11144* (2024).
- [32] Robert L Williams, Douglas A Lawrence, et al. 2007. *Linear state-space control systems*. John Wiley & Sons.
- [33] Haixu Wu, Tengge Hu, Yong Liu, Hang Zhou, Jianmin Wang, and Mingsheng Long. 2023. TimesNet: Temporal 2D-Variation Modeling for General Time Series Analysis. In *ICLR*. 1–23.
- [34] Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. 2023. Gated linear attention transformers with hardware-efficient training. *arXiv preprint arXiv:2312.06635* (2023).

- [35] Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. 2023. Are transformers effective for time series forecasting?. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 37. 11121–11128.
- [36] Michael Zhang, Khaled K Saab, Michael Poli, Tri Dao, Karan Goel, and Christopher Ré. 2023. Effectively modeling time series with simple discrete state spaces. *arXiv preprint arXiv:2303.09489* (2023).
- [37] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, et al. 2021. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *AAAI Conference on Artificial Intelligence*, Vol. 35. 11106–11115.
- [38] Tian Zhou, Ziqing Ma, Qingsong Wen, Xue Wang, Liang Sun, and Rong Jin. 2022. FEDformer: Frequency Enhanced Decomposed Transformer for Long-term Series Forecasting. In *ICML*.